Chapitre

Premiers pas avec Python 1

1.1 L'environnement de développement

Un **environnement de développement** [1] est un ensemble d'outils qui permet d'augmenter la productivité des programmeurs. Un **programme**, ou **script** est une succession de définitions, d'expressions et de commandes. Les **instructions** [2]) sont exécutées par l'**interpréteur** Python dans ce que l'on appelle la **console** [3].

[1] en anglais *integrated development environment* (IDE)

[2] en anglais *statement*

[3] en anglais *shell*, contraction de *shell program*

Figure 1.1 – L'EDL en ligne de console.basthon.fr. La console est à droite, l'éditeur de script est à gauche avec une numératation des lignes. On utilise la touche Exécuter pour lancer le script.

Dans la console, le symbole >>> (différent selon les EDL) est un *shell prompt*[4]. Il indique que l'interpréteur est dans l'attente de nouvelles instructions. Vous n'avez pas à le rentrer dans la partie éditeur. Par la suite, la présence >>> indique qu'il s'agit d'instructions rentrées directement dans la console.

[4] to prompt : (of a computer) request input from (a user) "the online form prompts users for data"

1.2 Présentation de la pythonette

— Émulateurs pour smartphone : Android et Apple Store
— Simulateurs en ligne epsilon.nsi.xyz et numworks.com/fr/simulateur

Dans le menu calcul, découvrir l'utilisation des touches shift ALPHA alpha sto→ F x^y et copy ; var paste " .

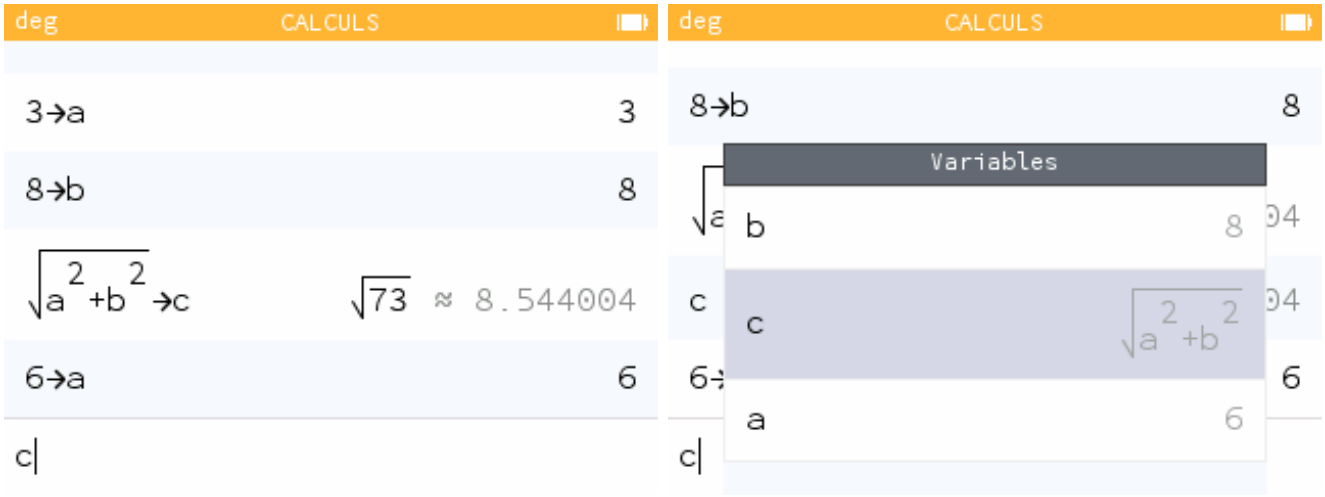


Figure 1.2 – Le menu calcul, expressions et stockage avec la touche sto→ F x^y

Dans le menu python, ouvrir un des scripts présent par défaut. Utiliser les touches shift et ◁ △ ▽ ▷ pour sélectionner un morceau de texte, puis couper/copier/coller/effacer avec les touches cut : x,n,t copy ; var paste " clear ⌫ .

Fermer le fichier, et ouvrir une console d'exécution.

Figure 1.3 – Utiliser paste " puis aller dans > modules > maths

```
>>> 3 * 2
6
>>> pi
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'pi' is not defined
```

Certaines instructions ne sont pas natives, vous devez les importer :

```
>>> from math import pi # importer l'instruction pi
>>> pi
3.141592653589793
>>> from math import *      # tout importer
>>> sqrt(2)                 # racine carrée
1.4142135623730951
```

Les type de valeurs

```
>>> type(4)
```

```
>>> type(4.0)
```

```
>>> type("4")
```

```
>>> type(3E5)
```

Les opérateurs Dans la console, tester les instructions suivantes et préciser l'opération effectuée :

```
>>> 32 + 10
```

```
>>> 80 - 38
```

```
>>> 6 * 7
```

```
>>> "bon" + "jour"
```

```
>>> 2 * "bon"
```

```
>>> "42" * 3
```

```
>>> 2 ** 6
```

```
>>> 6 ** 2
```

```
>>> 3 ** 2
```

```
>>> 42  / 6
```

```
>>> 42  / 11
```

```
>>> 42  / 0
```

```
>>> 42 // 11
```

```
>>> 42 // 6
```

```
>>> 42 % 11
```

```
>>> 42 % 6
```

Les opérateurs d'affectation Exécuter le script puis remplir le tableau :

```
>>> # à faire dans cet ordre
>>> a = 101
>>> b = 4
>>> e = a * b
>>> d = e + 2**8 + 6
>>> a = a - 1
>>> b = b + 2
```

variable	valeur finale
a	
b	
d	
e	

Exercice 1 Chercher la signification informatique des mots anglais *integer* et *string*.

Exercice 2 — exercices 10 à 13 page 9.

Exercice 3 — `https://bit.ly/2YiTCf3`.

```
a = 5
b = 6
a , b = b , a
```

1/ Qu'affiche ce script ?	6, 6	5 , 5	5, 6	6, 5

```
x , y = 1, 10    # double affectaction
c = y
y = x
x = c
print(x,y)
```

2/ Qu'affiche ce script ?	10 , 10	1 , 1	1 , 10	10 , 1

```
x=5
y=x+14
y=y*x
y=y+49
print(y)
```

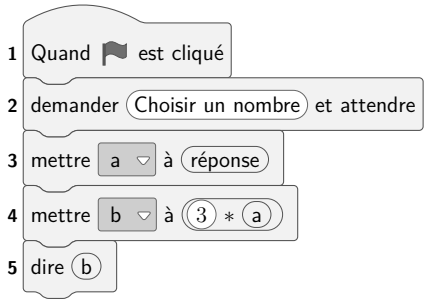
3/ Qu'affiche ce script ?	49	144	54	74

```
x = 7
y = 2 * x - 1
x = x + 3 * y
print(x , y)
```

4/ Qu'affiche ce script ?	130 , 13	13 , 130	46, 13	13, 46

```
a , b = 2 , 3 # double affectation
a = a ** b
a = a - b
b = a - b
print(a, b)
```

5/ Qu'affiche ce script ?	5 , 5	3 , 3	5 , 2	3 , 2




```
a = input("choisir un nombre...")
b = 3 * a
print(b)
```

6/ Qu'affiche ce script si on saisit 5 ?	15	15.0	555	message erreur

1.3 Fonctions

Une fonction est un sous programme que l'on peut appeler et exécuter plusieurs fois.

■ **Exemple 1.1 — Un premier exemple.** lien basthon notebook ou console

```
def mafonctionA (  a ) :
    b = a**2 - 3 * a + 2
    return b
def mafonctionB ( a ) :
    b = a**2 + 3 * a + 2
    print( b )
```

1/ L'appel `mafonctionA(1)` retourne	0	0.0	None	rien
2/ L'appel `mafonctionA(2.0)` retourne	0	0.0	None	rien
3/ L'appel `mafonctionB(-1.0)` retourne	0	0.0	None	rien

```
from math import sqrt
def hypotenuse(a, b):
    """théorème de Pythagore"""
    c = sqrt(a ** 2 + b ** 2)
    return c
```

4/ L'appel `hypotenuse(12,5)` retourne	13	13.0	None	42

```
def mafonctionC(b , a) :
    return 2 * a + b
c = mafonctionC(2**3 ,2+3 )
```

5/ L'appel `mafonctionC(3 , 2)` retourne	8	8.0	7	7.0
6/ L'appel `mafonctionC(2 , 3)` retourne	8	8.0	7	7.0
7/ La variable `c` vaut	16	17	18	21

```
def salutation() :
    return "Sire !"
    return "Bonjour !"
```

7/ L'appel `salutation()` retourne	"Bonjour !"	"Sire !"	"Sire !" puis "Bonjour !"	"Bonjour !" puis "Sire !"

Exercice 1 — exercices 14 (15, 17), 18, 22 (23) pages 11 à 13.

Exercice 2 lien baston

La fonction suivante calcule le montant de l'impôt d'une personne seule connaissant le montant de son revenu annuel.

```
def impotrevenu(r) :
    """r est le revenu annuel"""
    if r <= 10084 :
        impot = 0
    elif r <= 25710 :
        impot = 0.11*(r-10084)
    elif r <= 73516 :
        impot = 0.30*(r-25710)+impotrevenu(25710)
    elif r <= 158122 :
        impot = 0.41*(r-73516)+impotrevenu(73516)
    else :
        impot = 0.45*(r-158122)+impotrevenu(158122)
    return impot
```

a) Quelle valeur retourne `impotrevenu(10084)` ?

b) Quelle valeur retourne `impotrevenu(25710)` ?

c) Quelle valeur retourne `impotrevenu(30000)` ?

d) Rentrer le script sur la pythonette dans un script nommé `p01.py` et vérifiez le fonctionnement à l'aide de l'instruction `impotrevenu(30000)` dans la console. [5]

[5] Validation par le professeur ❐

e) Le taux d'imposition d'une personne est le ratio de l'impôt payé sur le revenu annuel.
Déterminer le taux d'imposition d'une personne seule avec 100000 € de revenu annuel.

1.4 Instructions conditionnelles

■ **Exemple 1.2**

```
1 chaine = "bateau"
2 if len(chaine) < 6 :
3     print("trop court")
4 print("fini avec la condition")
```

Le test à la ligne 2 vérifie si _____

Comme la condition est _____, le programme affiche _____puis procède à la ligne 4.

```
1 x = 15926354
2 if x % 6 == 0 :
3     print("multiple de ...")
4 else:
5     print("pas un multiple de ...")
6 print("fin du test")
```

Le test à la ligne 2 vérifie si _____

Comme la condition est _____, le programme va à la ligne _____.

Le programme affiche _____puis procède à la ligne _____ et affiche .

Exercice 1

```
1 a , b = 5 , 9
2 if a>6:
3     b=b-5
4 if b>=13:
5     b=b+9
```

1/ Code exécute lignes	2	3	4	5
2/ En fin de script b=	4	9	13	18

```
1 a , b = 10 , 19
2 if a>6:
3     b=b-4
4 if b>=14:
5     b=b+7
```

3/ Code exécute lignes	2	3	4	5
4/ En fin de script b=	15	19	22	26

```
1 a , b = 4 , 15
2 if a > 9 :
3     b = b-4
4 if b >= 12:
5     b = b+5
```

5/ Code exécute ligne 1	2	3	4	5
6/ En fin de script b=	11	15	16	20

```
1 a , b = 9 , 20
2 if a > 7 :
3     b = b-10
4 if b >= 12:
5     b = b+4
```

7/ Code exécute ligne 1	2	3	4	5
8/ En fin de script b=	10	14	20	24

```
1 a , b = 5 , 18
2 if a > 7 :
3     b = b-10
4 if b >= 12:
5     b = b+4
```

9/ Code exécute lignes	2	3	4	5
10/ En fin de script b=	8	12	18	22

```
1 a , b = 5 , 18
2 if a > 7 :
3     b = b-10
4     if b >= 12:
5         b = b+4
```

11/ Code exécute lignes	2	3	4	5
12/ En fin de script b=	8	12	18	22

```
1 a , b = 5 , 9
2 if a>=0 :
3     b=b-7
4 else :
5     b=b+7
6 if b>0 :
7     a=a+3
8 else :
9     a=a-3
```

13/ Code exécute lignes	3	5	7	9
14/ En fin de script a=	2	5	8	9

```
1 a , b = 7 , 0
2 if a >= 0 :
3     b = b - 7
4 else:
5     b = b + 7
6 if b > 0 :
7     a = a + 9
8 else:
9     a = a - 9
```

17/ Code exécute lignes	3	5	7	9
18/ En fin de script a=	-7	-2	7	16

```
1 a , b = -10 , -15
2 if a >= 0 :
3     b = b - 2
4 else:
5     b = b + 2
6 if b > 0 :
7     a = a + 6
8 else:
9     a = a - 6
```

15/ Code exécute lignes	3	5	7
16/ En fin de script a=	-17	-16	-13

```
1 a , b = 10 , 6
2 if a % 2 == 1 :
3     b = b - 7
4 else:
5     b = b + 7
6 if b % 3 == 2 :
7     a = a + 9
8 else :
9     a = a - 9
```

19/ Code exécute lignes	3	5	7	9
20/ En fin de script a=	1	6	10	19

Exercice 2

```
1 def funcA(arg1) :
2    if arg1 <= 0 :
3       return 0
4    else :
5       return 1
```

```
1 def funcB(arg1) :
2    if arg1 <= 0 :
3       return 0
4    else :
5       return arg1
```

```
1 def funcC(arg1) :
2    if arg1 % 2 == 0 :
3       return arg1//2
4    else :
5       return 3*arg1+1
```

1/ L'appel funcA(3) retourne	0	1	1.0	0.0
2/ L'appel funcA(0) retourne	0	1	1.0	0.0
3/ L'appel funcB(-5.0) retourne	0	-5	5	-5.0
4/ L'appel funcC(12) retourne	6	6.0	37	37.0
5/ L'appel funcC(9) retourne	4.5	27	28	28.0

Exercice 3 — exercices 27, 30, 31, 28 (32) pages 16 et 17.

1.5 Découverte des boucles finies for

`range(fin)` retourne la progression croissante d'entiers consécutifs de `0` inclus à `fin` exclu.
`range(debut, fin)` retourne la progression croissante d'entiers consécutifs de `debut` inclus jusqu'à `fin` exclu.
`range(debut, fin, pas)` retourne la progression croissante d'entiers : `debut`; `debut+pas`; `debut+2*pas`; `debut+3*pas` ... strictement inférieurs à `fin`.

■ **Exemple 1.3** On peut parcourir les éléments d'un objet `range` à l'aide d'une boucle `for`.

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

```
>>> for x in range(3, 8):
...     print(x)
...
3
4
5
6
7
```

```
>>> for entier in range(-3,2) :
...     nbr = entier**2
...     print(nbr)
...
9
4
1
0
1
```

Exercice 1 — exercices 34, 36 ,39 pages 19 à 23.

Exercice 2

Script A

```
n = 10
compteur = 0
for i in range(1,nbr) :
    if  n % i == 0 :
        print(i)
        compteur = compteur + 1
```

Nombre de boucles exécutées =
Script A affiche :
En fin de script A, `compteur`=

Script B

```
n = 11
compteur = 0
for i in range(1,n) :
    if  n % i == 0 :
        print(i)
    compteur = compteur + 1
```

Nombre de boucles exécutées =
Script B affiche :
En fin de script B, `compteur`=

Script C

```
n = 12
compteur = 0
for i in range(1,n) :
    if  n % i == 0 :
        print(i)
compteur = compteur + 1
```

Nombre de boucles exécutées =
Script C affiche :
En fin de script C, `compteur`=

■ **Exemple 1.4 — Je fais : principe accumulateur pour calculer une somme ou un produit.**

```python
def calculA() :
    nbr = 0
    compteur = 0
    for i in range(100) :
        nbr = nbr + i
        compteur = compteur + 1
    return nbr, compteur
```

```python
def calculB() :
    nbr = 0
    compteur = 0
    for i in range(5,15) :
        nbr = nbr + i**2
        compteur = compteur + 1
    return nbr, compteur
```

1) L'appel `calculA()` exécute ..... boucles, `calculB()` exécute ..... boucles
2) La valeur `nbr` retournée par `calculA()` correspond à la somme :

$$+ \quad + \quad + \quad + \quad + \ldots + \quad =$$

3) La valeur `nbr` retournée par `calculB()` correspond à la somme :

$$+ \quad + \quad + \quad + \quad + \ldots + \quad =$$

Exercice 3 — à vous.

```python
def calculC() :
    nbr = 0
    for i in range(5,10) :
        nbr = nbr + i**3
    return nbr
```

```python
def calculD() :
    nbr = 1
    for i in range(2,6) :
        nbr = nbr * i
    return nbr
```

1) L'appel `calculC()` retourne :

$$+ \quad + \quad + \quad + \quad + \ldots + \quad =$$

2) L'appel `calculD()` retourne :

```python
def calculE(n) :
    nbr = 0
    for i in range(1, n) :
        nbr = nbr + 2**(-i)
    return nbr
```

```python
def calculF(n) :
    nbr = 0
    for i in range(n+1) :
        nbr = nbr + 4**(-i)
    return nbr
```

3) Les appels `calculE(10)` et `calculF(10)` retournent le résultat des calculs :

Exercice 4 — exercices 43 et 44 page 23.

1.6 Correction et écriture de scripts

Exercice 1 Les scripts ci-dessous doivent retourner respectivement l'aire du rectangle de dimensions `longueur` et `largeur` et l' aire du disque de rayon nommé `rayon`. Corriger les erreurs de chaque fonctions (5 au total).

```
def rectangle(longueur, largeur) :
    return aire

def fonction (rayon)
aire = pi*r*2
return aire
```

Exercice 2 La fonction `functionB` prend deux arguments nommés `longueur` et `largeur` d'un rectangle, et retourne le périmètre du rectangle correspondant. On souhaite qu'elle retourne `False` si une des dimensions en argument est négative.

Corriger les 6 erreurs de ce script :

```
def fonction (longueur; largeur)
    if longueur < 0
        return False
    if largeur < 0
        return True
    perimetre=2*longueur+2*largeur
        return perimetre
```

Exercice 3 Écrit le script d'une fonction d'appel `functionC` qui prend un argument nommé `arg` et qui retourne la somme de son carré et de 5.

```
1
2
3
4
5
6
7
```

Exercice 4 Écrit le script d'une fonction d'appel `functionD` qui prend un argument nommé `arg` et qui retourne son carré s'il est positif, et l'opposé du carré s'il est négatif ou nul.

```
1
2
3
4
5
6
7
```

Exercice 5 Écrit le script d'une fonction d'appel `functionE` qui prend un argument nommé `arg` et qui retourne sa racine carrée s'il est positif ou nul, et zéro s'il est négatif.

```
1
2
3
4
5
6
7
```

Exercice 6 Écrit le script d'une fonction d'appel `functionF` qui prend deux arguments nommés `arg1` et `arg2` et qui retourne `arg2` si `arg1` est positif ou nul, et le triple de `arg2` si `arg1` est négatif strictement.

```
1
2
3
4
5
6
7
```

1.7 Découverte des boucles infinies while

Une boucle infinie `while` est exécutée **tant que** la condition est `True`.
Sa forme générale est :

```
while test == True :
    """une ou plusieurs lignes d'instructions de la
   boucle avec indentation"""
"""sortie de la boucle si test  == False"""
```

■ **Exemple 1.5**

```
i = 5
while i!=0 :
    print(i)
    i = i - 1
print(f"sortie i={i}")
```

5 4 3 2 1 sortie i=0
En sortie de boucle

```
i = 2
while 91 % i !=0 :
    print(i)
    i = i + 1
print(f"sortie i={i}")
```

2 3 4 5 6 sortie i=7
En sortie de boucle

```
i = 10
while i**2 < 1664 :
    i = i + 1
print(f"sortie i={i}")
```

sortie i=41

■ **Exemple 1.6** En vous aidant du tableau, préciser ce qu'affiche le script suivant :

```
n=1
while 1.11**n < 2 :
    n = n+1
print(n)
```

n	1	2	3	4	5	6	7	8
1.11**n								

- ❏ Tous les entiers tels que $1.11^n < 2$
- ❏ Le plus grand entier n tel que $1.11^n \geqslant 2$
- ❏ Le plus petit entier n tel que $1.11^n \geqslant 2$
- ❏ Rien car il ne s'arrête pas
- ❏ 2.0761602
- ❏ 7

Exercice 1 — à vous. Qu'affichent les programmes suivants. Cochez la bonne réponse.

```
n=1
while 0.88**n > 0.5 :
    n = n+1
print(n)
```

n	2	3	4	5	6	7	8
0.88**n							

- ❏ 6
- ❏ 5
- ❏ Le plus petit entier n tel que $0.88^n \geqslant 0.5$
- ❏ Le plus petit entier n tel que $0.88^n \leqslant 0.5$
- ❏ Rien car il ne s'arrête pas
- ❏ 0.464404

```
n=1
while 1.15**n < 2 :
    n = n+1
print(n)
```

n	2	3	4	5	6	7	8
1.15**n							

- ❏ 5
- ❏ Le plus petit entier n tel que $1.15^n \geqslant 2$
- ❏ Rien car il ne s'arrête pas
- ❏ 2,011 357 2

1.8 TP Aliens

Partie A

— Scripts 1 à 8 : variables
— Scripts 9 à 20 : boucles for

Partie B

— Scripts 21 à 26 : conditionnelles
— Scripts 27 à 35 : conditionnelles et boucles for

Partie C

— Scripts 36 à 47 : boucles infinies while

Partie D

— Scripts 48 à 56 : fonctions
— Scripts 57 à 62 : boucles for

Partie E Scripts A à F : écrire un script

1.9 TP Pyrates

`https://www.py-rates.fr` code de session .....

Refaire le niveau 1 comme montré puis un maximum parmi les 8 niveaux.

Notation :

— refaire le niveau 1 → 4/20
— faire le niveau 2 → 6/20
— faire le niveau 3 → 8/20
— faire le niveau 4 → 10/20
— faire le niveau 5 → 12/20
— faire le niveau 6 → 14/20
— faire le niveau 7 → 16/20
— faire le niveau 8 → 20/20

1.10 Le Module Turtle par l'exemple

```
from turtle import *
penup()
pas = 20
goto(0,0)
setheading(0)
pendown()
for i in range(4) :
    forward(pas)
    left(90)
    forward( 2*pas )
    goto(0,0)
    pas = pas + 20
penup()
```

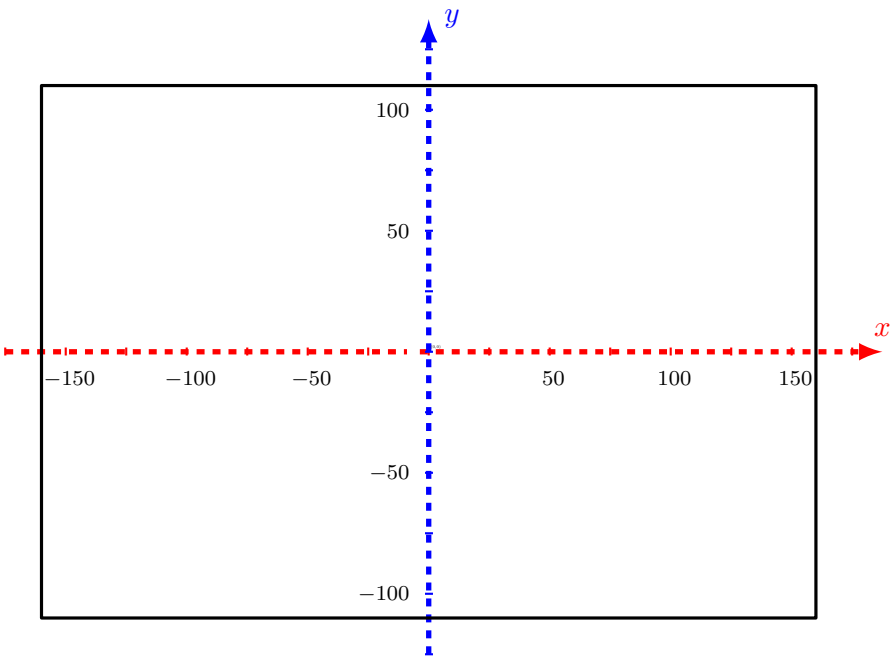


Figure 1.4 – Les coordonnées de l'écran par défaut vont de -160 à $+160$ pour les x et de -110 à $+110$ pour les y ; $(0, 0)$ est au centre de l'écran.

DM Les mathématiques sont belles ! Réaliser avec votre calculatrice une belle construction soit en écrivant un algorithme python qui exploite le module turtle ou le module kandinsky.

Consignes :

1. Le travail doit être réalisé sur votre calculatrice, la NumWorks. Si vous n'avez pas cette calculatrice, vous pourrez utiliser l'émulateur en ligne, il est gratuit.
2. N'hésitez pas à consulter des exemples de réalisations, vous verrez ainsi ce qu'il est possible de réaliser. Copier un travail déjà fait est une très mauvaise idée.
3. Vous pouvez également faire des recherches sur internet, mais là encore copier un script sans faire aucune modification est risqué.
4. Vos meilleures réalisations seront soumises à NumWorks. Des coques personnalisée pour leur calculatrice sont à gagner pour des concours régulier.
5. Vous devez conserver une trace de vos recherches sur internet et des ressources que vous avez exploitées, elles vous seront demandées dans le compte rendu électronique. (Sur un ordinateur PC, appuyez sur CTRL + H pour récupérer l'historique de votre navigateur web et consulter cette page pour retrouver l'historique associé à votre compte google.)
6. Vous pouvez faire les essais sur votre calculatrice, et avancer ce travail pendant vos heures d'études par exemple, mais il sera impératif de le finaliser sur l'espace « Mes scripts » depuis `https://my.numworks.com/python/` . Il faut ici aussi de connecter avec son compte NumWorks puis se rendre dans « mes scripts »
   Laissez bien la case « Permettre aux autres utilisateurs d'accéder à ce script » coché sinon il sera impossible de voir le travail rendu. Le script sera nommé dmSNT.py et vous pouvez écrire ce que vous voulez dans le descriptif. Comme le script est public, il est conseillé d'éviter de mettre son identité complète.
   Il n'y pas de bouton "Capture d'écran", aussi vous devez procéder ainsi. 1. Lancer le script 2. Clic droit sur l'écran de la calculatrice > Enregistrer l'image sous Une image propre sans bordure sera générée, comme celle-ci obtenue à partir de ce script :
   https://my.numworks.com/python/cent20/dmSNT
7. L'image et le lien hypertexte devront être rendus via un formulaire électronique